

Algorithm Masterclass: Two Pointers

Optimizing $O(n^2)$ Traversals to $O(n)$

Presented by: QMML Society

Introduction to Two Pointers

The **two-pointer approach** is an algorithmic technique that uses two indices (or "pointers") to traverse a data structure (array, string, linked list) simultaneously. By coordinating these two positions, you can solve problems in a single pass ($O(n)$ time), replacing brute-force nested loops ($O(n^2)$).

Common Variations:

1. **Opposite Ends (Converging)**: Pointers start at $left = 0$ and $right = n - 1$ and move toward each other. Great for sorted arrays, palindromes, and reversing.
2. **Same Direction (Fast & Slow)**: Both start at 0 but move at different interval speeds. Great for Linked List cycle detection.
3. **Sliding Window**: Pointers define an expanding/shrinking "window" of elements.

Use this approach when the input is sorted, searching for valid pairs, or needing to optimize $O(n^2)$ down exactly to $O(n)$ memory-lessly ($O(1)$ space).

Valid Palindrome

Problem Statement: Given a string `s`, determine if it is a palindrome. A palindrome is a string that reads the same forward and backward.

- The check must be **case-insensitive**.
- You must **ignore all non-alphanumeric characters** (spaces, punctuation).

Example:

- Input: `"A man, a plan, a canal: Panama"`
- Processed internally: `"amanaplanacanalpanama"`
- Output: `true`

Core Concepts: String Manipulation

Before jumping to the two pointers, we need to know how to handle Strings.

Strings in both Java and Python are **Immutable**! Operations that seem to modify a string actually spawn entirely new strings in memory.

Java Tools

- `Character.isLetterOrDigit(c)` : Checks if a char is alphanumeric.
- `Character.toLowerCase(c)` : Safely downcases a character.
- `StringBuilder` : Essential for modifying/building strings safely without causing massive memory overhead from the `+` operator.

Python Tools

- `char.isalnum()` : Checks if a character is alphanumeric.
- `char.lower()` : Converts string to lowercase.
- `"".join(list)` : The most efficient way to build strings from character lists.
- `s[::-1]` : Pythonic syntactic sugar for effortlessly reversing a string.

Valid Palindrome: Brute Force Approach

The brute-force way is simple: Clean it, Reverse it, Compare it.

Algorithm

1. **Clean**: Create an empty temporary buffer. Iterate through every character in the original string. If it's alphanumeric, convert it to lowercase and append it.
2. **Reverse**: Create a reversed copy of the new freshly cleaned string.
3. **Compare**: Check if `cleaned_string == reversed_string`.

Complexity

- **Time**: $O(n)$ - Traversing the string, reversing it, and comparing it all take linear steps.
- **Space Complexity**: $O(n)$ - We are physically allocating memory to create the entirely brand new `cleaned_string` and `reversed_string` objects!

Brute Force Valid Palindrome: Java & Python

```
// JAVA: O(N) Space Complexity
class Solution {
    public boolean isPalindrome(String s) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isLetterOrDigit(c)) { sb.append(Character.toLowerCase(c)); }
        }
        String cleaned = sb.toString();
        String reversed = new StringBuilder(cleaned).reverse().toString();

        return cleaned.equals(reversed);
    }
}
```

```
# PYTHON: O(N) Space Complexity
class Solution:
    def isPalindrome(self, s: str) -> bool:
        cleaned_chars = []
        for char in s:
            if char.isalnum(): cleaned_chars.append(char.lower())

        cleaned_s = "".join(cleaned_chars)
        reversed_s = cleaned_s[::-1] # Pythonic Slice Reversal
        return cleaned_s == reversed_s
```

Valid Palindrome: Optimal Two-Pointers

To achieve peak memory efficiency, we absolutely **cannot allocate new strings**. We must validate it directly *in-place* on the original string memory using Two Pointers.

Algorithm

1. Initialize `left` at 0 and `right` at `s.length() - 1`.
2. **Loop** while `left < right` :
 - If `s[left]` is NOT alphanumeric, just do `left++` and skip it!
 - If `s[right]` is NOT alphanumeric, just do `right--` and skip it!
3. Now both pointers are sitting on valid letters. Compare their lowercase forms!
 - If they **don't match**: `return false` immediately!
 - If they **match**: Move both pointers inward (`left++` , `right--`) and repeat.
4. If the pointers safely cross without triggering false, `return true` !

Complexities: Time: $O(n)$, Space: $O(1)$!

Valid Palindrome: Pseudocode

```
function isPalindrome(string s):  
    left = 0  
    right = length(s) - 1  
  
    while left < right:  
        while s[left] is not alphanumeric and left < right:  
            left = left + 1  
  
        while s[right] is not alphanumeric and left < right:  
            right = right - 1  
  
        if lowercase(s[left]) != lowercase(s[right]):  
            return false  
  
        left = left + 1  
        right = right - 1  
  
    return true
```

Optimal Valid Palindrome: Java Implementation

```
class Solution {
    public boolean isPalindrome(String s) {
        int left = 0;
        int right = s.length() - 1;
        while(left < right){
            while(!Character.isLetterOrDigit(s.charAt(left))){
                left++;
            }
            while(!Character.isLetterOrDigit(s.charAt(right))){
                right--;
            }
            if(s.toLowerCase().charAt(left) != s.toLowerCase().charAt(right)){
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}
```

Optimal Valid Palindrome: Python Implementation

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        left, right = 0, len(s) - 1

        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1

            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1

            if left < right:
                # Compare the lowercase representations directly mapped in memory
                if s[left].lower() != s[right].lower():
                    return False

            # Successful match => converge window
            left += 1
            right -= 1

        return True
```

Two Sum II – Input array is sorted

Problem Statement:

Given a **1-indexed** array of integers **numbers** that is *already sorted* in non-decreasing order, find two numbers that sum up to a specific **target** number.

- Return exactly their 1-indexed positions **[index1, index2]**.
- Exactly one solution exists.
- **Cannot** use the exact same element twice.

Example:

- Input: **numbers = [2, 7, 11, 15]**, **target = 9**
- Output: **[1, 2]**
- Explanation: **2** is at 1-index 1, **7** is at 1-index 2. They sum to 9!

Two Sum II: Hash Map Approach

Since this is a variant of the original *Two Sum* problem, our instinct is to use a Hash Map.

Algorithm

1. Initialize a `HashMap` to store `{number : 1-indexed_position}`.
2. Iterate through the array. For every `currentNum` :
 - Calculate what we need: `complement = target - currentNum`.
 - If `complement` exists in our map, we found our pair! Return `[map.get(complement), currentIndex + 1]`.
 - If not, store the `currentNum` and its `currentIndex + 1` into the Map so future numbers can find it.

Complexity

- **Time:** $O(n)$ - Constant time map lookups.
- **Space Complexity:** $O(n)$ - We are using heavy memory allocations to build out the massive Hash Map data structure.
Can we do better? Yes, because the array is sorted!

Two Sum II: Optimal Two Pointers Approach

Because the input array is **already sorted**, we don't need to waste $O(n)$ memory building a Hash Map! We can cleanly slice through using Two Pointers!

Algorithm

1. Set `left = 0` (smallest value) and `right = length - 1` (largest value).
2. Calculate the `current_sum = numbers[left] + numbers[right]`.
 - **Case 1:** `current_sum == target` → Return `[left + 1, right + 1]`.
 - **Case 2:** `current_sum < target` → The sum is currently too small. We must aggressively increase our sum. Since it's sorted, we move `left++` !
 - **Case 3:** `current_sum > target` → The sum is currently too huge. We must drop our value safely. Since it's sorted, we move `right--` !

Complexity

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$ - Peak perfection!

Two Sum II: Pseudocode

```
function twoSum(array numbers, integer target):  
    left = 0  
    right = length(numbers) - 1  
  
    while left < right:  
        current_sum = numbers[left] + numbers[right]  
  
        if current_sum == target:  
            return [left + 1, right + 1] // 1-indexed output  
  
        else if current_sum < target:  
            left = left + 1 // Need larger numbers  
  
        else:  
            right = right - 1 // Need smaller numbers  
  
    return [0, 0]
```

Optimal Two Sum II: Java Implementation

```
// ZERO extra memory utilization: purely manipulating memory pointers!
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int left = 0;
        int right = numbers.length - 1;
        while(left < right){
            int sum = numbers[left] + numbers[right];
            if(target == sum) return new int[]{left + 1, right + 1};
            if(target < sum) right--;
            if(target > sum) left++;
        }
        return new int[]{0,0};
    }
}
```

Optimal Two Sum II: Python Implementation

```
# Utilizing the strictly sorted nature to natively pinpoint the target sum
class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        left = 0
        right = len(numbers) - 1

        while left < right:
            current_sum = numbers[left] + numbers[right]

            if current_sum == target:
                # Add 1 to convert structural 0-index to required 1-index output
                return [left + 1, right + 1]

            elif current_sum < target:
                # Sum too weak: push lower bound higher
                left += 1

            else:
                # Sum too strong: pull upper bound lower
                right -= 1

        return [0, 0]
```

3Sum: Problem Statement

Problem: Find all unique triplets `[nums[i], nums[j], nums[k]]` in an integer array `nums` such that their sum is zero:

`nums[i] + nums[j] + nums[k] == 0`.

- Indices `i`, `j`, and `k` must be distinct.
- The output must **not** contain duplicate triplets (e.g., `[-1, 0, 1]` is the same as `[0, -1, 1]`).

Core Concepts to Master:

1. **Sorting** (`Arrays.sort()` / `list.sort()`): Critical for efficiently skipping duplicates and enabling the Two-Pointer approach.
2. **Hash Sets** (`HashSet` / `set`): Excellent for grabbing unique items in $O(1)$ time.
3. **Two-Pointer technique**: Reduces $O(n^3)$ triple-nested loops down to $O(n^2)$ by locking one number and using two pointers to scan the rest!

3Sum: Brute-Force Approach

The logic: Check every possible combination of 3 distinct numbers.

Algorithm

1. Initialize an empty `HashSet` to auto-filter duplicate triplets.
2. Use **three nested loops** (`i`, `j`, `k`).
3. If `nums[i] + nums[j] + nums[k] == 0` :
 - Sort the triplet so `[-1, 0, 1]` and `[1, 0, -1]` both become `[-1, 0, 1]`.
 - Add it to the `HashSet`.
4. Convert the `HashSet` to a List to return.

Complexity

- **Time:** $O(n^3)$ - Running three nested loops will trigger a Time Limit Exceeded (TLE) on LeetCode for larger arrays.
- **Space:** $O(M)$ to store the triplets.

3Sum: Optimal Algorithm (Sorting & Pointers)

1. **Sort the array** ($O(n \log n)$). Sorting natively groups duplicates together!
2. **Iterate** using **i** from **0** to **n-3**:
 - *Optimization 1*: If **nums[i] > 0**, break! Because the array is sorted, there's no way three positive numbers can sum to 0.
 - *Optimization 2*: If **nums[i] == nums[i-1]**, skip to avoid duplicate triplets!
3. **Lock nums[i]** and use Two Pointers for the remaining array:
 - **left = i + 1**, **right = n - 1**, **target = -nums[i]**.
 - Run the standard Two Sum II approach!
 - *Optimization 3*: When a valid triplet is found, push **left++** and **right--** until they land on new, non-duplicate numbers.

Complexity: Time: $O(n^2)$ (Outer loop is $O(n)$ $\times$ inner Two-Pointer loop is $O(n)$). Space: $O(1)$.

3Sum: Pseudocode

```
function threeSum(array nums):
    sort(nums)
    result = []

    for i from 0 to length(nums) - 3:
        if i > 0 and nums[i] == nums[i - 1]: continue // Skip duplicates
        if nums[i] > 0: break // Optimization: array is sorted!

        left = i + 1
        right = length(nums) - 1
        target = -nums[i]

        while left < right:
            current_sum = nums[left] + nums[right]
            if current_sum == target:
                result.add([nums[i], nums[left], nums[right]])
                left++, right--
                // Skip inner duplicates
                while nums[left] == nums[left - 1]: left++
                while nums[right] == nums[right + 1]: right--
            else if current_sum < target:
                left++
            else:
                right--

    return result
```

Optimal 3Sum: Java Implementation

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(nums); // 1. Sort!

        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip duplicates for i
            if (nums[i] > 0) break; // Optimization

            int left = i + 1, right = nums.length - 1;
            int target = -nums[i];

            while (left < right) {
                if (nums[left] + nums[right] == target) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    left++; right--;
                    // Skip duplicates for left and right!
                    while (left < right && nums[left] == nums[left - 1]) left++;
                    while (left < right && nums[right] == nums[right + 1]) right--;
                } else if (nums[left] + nums[right] < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

Optimal 3Sum: Python Implementation

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        result = []
        nums.sort() # 1. Sort!

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]: continue # Skip duplicates
            if nums[i] > 0: break # Optimization

            left, right = i + 1, len(nums) - 1
            target = -nums[i]

            while left < right:
                current_sum = nums[left] + nums[right]
                if current_sum == target:
                    result.append([nums[i], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    # Skip duplicates for left and right
                    while left < right and nums[left] == nums[left - 1]: left += 1
                    while left < right and nums[right] == nums[right + 1]: right -= 1
                elif current_sum < target:
                    left += 1
                else:
                    right -= 1

        return result
```

Container With Most Water

Problem: You have a series of vertical lines on an x-axis. `heights[i]` represents the height of the i -th line. Find two lines that form a container capable of holding the absolute maximum amount of water between them.

Core Math:

- The distance between the lines creates the **width**.
- The container can only hold water up to the **shorter** line.
- $\text{Area} = \min(\text{height}[i], \text{height}[j]) \times (j - i)$

Brute-Force Approach:

Use nested loops to pair every single line with every other line, tracking the max area.

- Time Complexity: $O(n^2)$
- This works, but we can instantly do better by realizing width is constantly decaying.

Container With Most Water: The Intuition

Why does Two Pointers work here in $O(n)$?

1. If we place our pointers at the absolute edges (`left = 0` , `right = n-1`), we guarantee the **maximum possible width**.
2. To improve the area now, we *must* find a taller line to compensate for the decaying width as we move inwards!
3. If we move the taller line inwards, we are strictly losing width without gaining any height (because the container is bottlenecked by the short line).
4. **Conclusion**: Always discard the bottleneck! Move the pointer of the explicitly **shorter line** inwards, hoping to strike a massively taller pillar!

Container With Most Water: Pseudocode

```
function maxArea(array heights):
    left = 0
    right = length(heights) - 1
    max_area = 0

    while left < right:
        width = right - left
        height_limit = minimum(heights[left], heights[right])
        current_area = width * height_limit

        max_area = maximum(max_area, current_area)

        // Discard the bottleneck boundary
        if heights[left] < heights[right]:
            left = left + 1
        else:
            right = right - 1

    return max_area
```

Optimal Container With Most Water: Java

```
class Solution {
    public int maxArea(int[] heights) {
        int left = 0;
        int right = heights.length - 1;
        int maxArea = 0;

        while (left < right) {
            // Calculate current container potential
            int currentHeight = Math.min(heights[left], heights[right]);
            int currentWidth = right - left;
            int currentArea = currentHeight * currentWidth;

            maxArea = Math.max(maxArea, currentArea);

            // Discard the bottleneck line
            if (heights[left] < heights[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxArea;
    }
}
```

Optimal Container With Most Water: Python

```
class Solution:
    def maxArea(self, heights: list[int]) -> int:
        left, right = 0, len(heights) - 1
        max_area = 0

        while left < right:
            current_height = min(heights[left], heights[right])
            current_width = right - left
            current_area = current_height * current_width

            max_area = max(max_area, current_area)

            # Evolve the window by ditching the limiting factor
            if heights[left] < heights[right]:
                left += 1
            else:
                right -= 1

        return max_area
```

Trapping Rain Water

Problem: Given an array representing an elevation map of bar heights, calculate the maximum area of water structurally trapped in the dips *after it rains*.

- Unlike the previous problem, water here is dynamically trapped between topological peaks, not just bounded by 2 specific walls.

The Logic:

The water level floating above any specific bar **i** is strictly determined by:
 $\min(\text{Tallest Bar to Left}, \text{Tallest Bar to Right}) - \text{height}[i]$

Brute-Force Approach:

For *every single bar*, iterate $O(n)$ left to find the max, and $O(n)$ right to find the max.

- Time Complexity: $O(n^2)$.

Trapping Rain Water: Two Pointer Logic

We don't actually need to know the *exact* left and right absolute maximums. We only care about the **Bottleneck**.

1. Place pointers at `left = 0` and `right = n-1`.
2. Maintain tracking variables `max_left` and `max_right`.
3. If `height[left] < height[right]`, we are guaranteed that `height[left]` is natively bottlenecked by the left side. A massive wall exists on the right side, so the limiting factor is simply `max_left`!
 - Process `left`: Water trapped = `max_left - height[left]`. Move `left++`.
4. If `height[left] >= height[right]`, the bottleneck logic flips.
 - Process `right`: Water trapped = `max_right - height[right]`. Move `right--`.

This collapses $O(n^2)$ down to a brilliantly clean $O(n)$ single-pass solution.

Trapping Rain Water: Pseudocode

```
function trap(array height):
    left = 0
    right = length(height) - 1
    max_left = 0
    max_right = 0
    total_water = 0

    while left <= right:
        if height[left] < height[right]:
            if height[left] >= max_left:
                max_left = height[left]
            else:
                total_water = total_water + (max_left - height[left])
                left++
        else:
            if height[right] >= max_right:
                max_right = height[right]
            else:
                total_water = total_water + (max_right - height[right])
                right--

    return total_water
```

Optimal Trapping Rain Water: Java

```
class Solution {
    public int trap(int[] height) {
        if (height == null || height.length < 3) return 0;

        int left = 0, right = height.length - 1;
        int maxLeft = 0, maxRight = 0;
        int totalWater = 0;

        while (left <= right) {
            if (height[left] < height[right]) {
                if (height[left] >= maxLeft) maxLeft = height[left];
                else totalWater += maxLeft - height[left];
                left++;
            } else {
                if (height[right] >= maxRight) maxRight = height[right];
                else totalWater += maxRight - height[right];
                right--;
            }
        }
        return totalWater;
    }
}
```

Optimal Trapping Rain Water: Python

```
class Solution:
    def trap(self, height: list[int]) -> int:
        if not height or len(height) < 3: return 0

        left, right = 0, len(height) - 1
        max_left, max_right = 0, 0
        total_water = 0

        while left <= right:
            if height[left] < height[right]:
                if height[left] >= max_left: max_left = height[left]
                else: total_water += max_left - height[left]
                left += 1
            else:
                if height[right] >= max_right: max_right = height[right]
                else: total_water += max_right - height[right]
                right -= 1

        return total_water
```

Wrap-Up: The Power of Pointers

By abandoning raw brute-force nesting, Two Pointers allows us to fluidly dance across arrays using mathematical truths:

- **Sorted Bounds:** When organized, pointers mathematically guarantee we move toward the target efficiently.
- **Bottleneck Discovery:** Whether expanding a physical container of water or trapping drops between peaks, securely tracking the absolute edge bottleneck solves spatial scaling natively in linear time!